



UNIVERSIDADE
AUTÓNOMA
DE LISBOA

RELATÓRIO DO PROJETO DE RECONHECIMENTO FACIAL

**Elementos: Eliandro Agostinho(30010598), Breno Lourenço(30011567), Lucas
Gomes(30006482)**

Relatório do Projeto: Reconhecimento Facial e Análise de Emoções

1. Objetivo do Projeto

Desenvolver um software em Python para reconhecer rostos em imagens e analisar as emoções dominantes, utilizando uma arquitetura bem estruturada baseada no padrão MVC (Model-View-Controller). O projeto deve ser modular, escalável e fácil de manter.

2. Estrutura do Projeto

O projeto foi organizado em uma estrutura de pastas seguindo o padrão MVC, da seguinte forma:

- Model: Contém os módulos responsáveis pela lógica principal do programa (reconhecimento facial e análise de emoções).
 - View: Gerencia a interface gráfica do usuário, implementada com a biblioteca PySide6.
 - Controller: Intermedia a comunicação entre o Model e a View.
 - main.py: Ponto de entrada do programa, responsável por inicializar o controlador e carregar a interface gráfica.
-

3. Componentes Implementados

3.1 Model

1. Emotion_Analysis.py

- Implementa o método analisar_emocao para identificar a emoção dominante em uma imagem usando a biblioteca DeepFace.
- Utiliza ações configuradas para a análise de emoções e retorna os resultados com nível de confiança.

2. Face_Recognition_Handler.py

- Contém dois métodos principais:
 - **treinar_reconhecedor:** Treina o modelo de reconhecimento facial utilizando o algoritmo LBPH (Local Binary Patterns Histogram) da biblioteca OpenCV.
 - **reconhecer_rosto:** Identifica um rosto em uma imagem fornecida e retorna a etiqueta e o nível de confiança.
-

3.2 Controller

1. Face_Recognition_Controller.py

- Gerencia a lógica de alto nível, conectando o modelo à interface gráfica.
 - Métodos implementados:
 - **treinar_modelo:** Chama a função de treinamento do modelo de reconhecimento facial.
 - **reconhecer:** Processa imagens para identificar rostos, verificando se o modelo foi previamente treinado.
 - **analisar_emocao:** Chama a função de análise de emoções do Model.
-

3.3 View

1. Face_Recognition_View.py

- Interface gráfica projetada com PySide6.
- Elementos principais:
 - Botões para carregar imagens, reconhecer rostos e analisar emoções.
 - Slider para controle de parâmetros adicionais (exemplo: ajustes no modelo).
- Funções associadas:
 - **browse_file:** Abre um diálogo para selecionar imagens.
 - **process_image:** Conecta a lógica de análise de emoções e reconhecimento facial ao controlador.

- `show_emotion` e `show_face_result`: Exibem os resultados ao usuário.
 - `show_error`: Exibe mensagens de erro quando necessário.
-

3.4 Main

1. `main.py`

- Inicializa o controlador e a interface gráfica.
 - Configura a aplicação para executar o programa principal com a estrutura MVC.
 - Exemplo de treinamento do modelo foi incluído para testes, com dados reais a serem fornecidos.
-

4. Problemas Enfrentados e Soluções

1. `ModuleNotFoundError` para Model, View ou Controller

- **Solução:** Implementado o arquivo `__init__.py` em cada pasta para transformar as pastas em pacotes Python e ajustar as importações relativas.

2. Erro ao instalar bibliotecas (cv2, numpy, PySide6)

- **Solução:** Foi necessário garantir que o ambiente Python estava configurado corretamente. Soluções incluíram:
 - Atualização do pip.
 - Verificação do ambiente virtual (venv).
 - Uso do comando correto para instalar pacotes: `pip install opencv-python-headless`, `pip install numpy`, `pip install PySide6`.

3. Estruturação do Projeto

- Foi debatido se o arquivo `main.py` deveria estar em uma subpasta. Decidiu-se que o mesmo ficaria fora das pastas do MVC, já que é o ponto central que conecta todos os componentes.

4. Falta de Imagens de Treinamento

- Criada a sugestão de um diretório específico, como `data/images`, para armazenar imagens de treinamento e teste, com um script para carregamento dinâmico.
-

5. Conclusões e Próximos Passos

Até o momento, o projeto foi estruturado corretamente e os principais componentes estão implementados. Os próximos passos incluem:

1. Treinamento e Teste do Modelo:

- Fornecer um conjunto de imagens para treinar o modelo de reconhecimento facial.
- Testar a análise de emoções e ajuste de parâmetros do DeepFace.

2. Melhorias na Interface Gráfica:

- Exibir os resultados diretamente na interface, em vez de imprimir no console.
- Adicionar suporte para múltiplas línguas (opcional).

3. Validação do Software:

- Garantir que todas as funções estejam funcionando corretamente, testando cenários reais com diferentes imagens e condições.

4. Documentação e Testes:

- Adicionar mais comentários no código para facilitar a manutenção.
- Criar testes unitários para cada módulo.

